

VLSI Project 2 Report

2022171005 권민찬

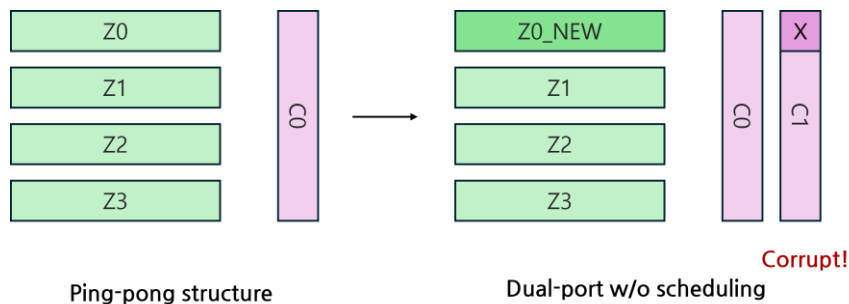
1. JPEG Conversion

용량이 큰 RAW 이미지를 메모리에 효율적으로 저장하기 위해서 사용하는 파일 형식이 JPEG이다. JPEG은 주파수 차이에 대한 인간 눈의 민감도 차이를 이용해 파일을 압축한다. 사람의 눈은 이미지의 밝기, 윤곽 등에 대응하는 저주파 성분에 민감하고, 미세한 질감이나 잔무늬에 대응하는 고주파 성분에는 훨씬 덜 민감하다. 2D-DCT 연산을 통해서 입력된 신호를 주파수 영역으로 변환하고 양자화를 통해 고주파 성분을 줄이고, 저주파 성분을 보존함으로써 이미지의 품질을 보존하면서 용량을 줄일 수 있게 된다.

이 프로젝트에서는 메모리에 저장된 이미지 파일을 읽어 2D-DCT를 수행하고, 다시 메모리에 쓰는 모듈을 설계한다. 이후 메모리에 쓰인 값을 읽어 MATLAB으로 다시 복원하여 PSNR을 측정함으로써 DCT 모듈이 잘 설계됐는지 검증할 수 있다. 모든 이미지의 PSNR 값이 29.0 이상이 되고, 면적을 최소화하기 위해 다양한 기법을 적용해서 모듈을 설계했다.

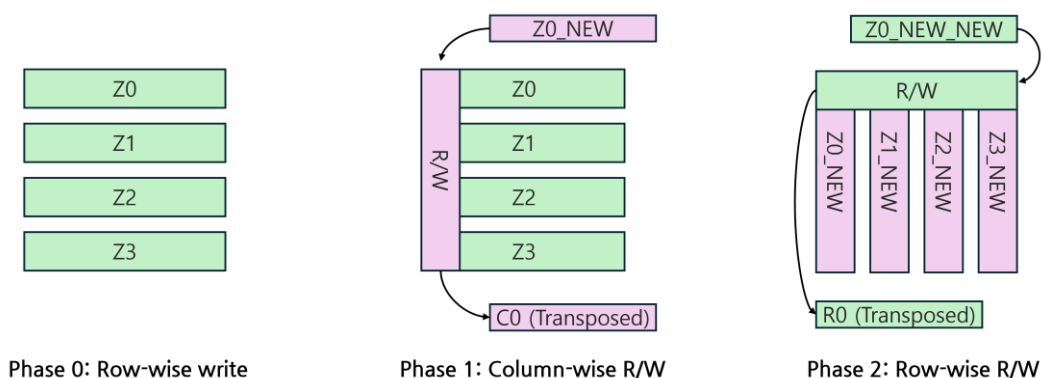
2. Techniques for Area Optimization

i. Efficient Scheduling for the Transposed Memory



RTL 작성을 하기 전에, 컨트롤 로직을 수정하는 것을 피하기 위해서 TPMEM의 개수를 줄일 수 있는 방법을 고민했다. TPMEM에서 데이터가 덮어쓰워지지 않고 열방향으로 읽기 위해서는 열 방향 읽기가 모두 끝날 때까지 쓰기가 일어나서는 안 된다. TPMEM에는 매 클럭마다 새로운 행 방향 데이터가 들어오는데, 데이터 덮어쓰기를 방지하고 throughput을 유지하기 위해 TPMEM을 핑퐁 구조로 두 개씩 사용해야 했다. 일단 듀얼 포트로 읽기와 쓰기가 동시에 가능하도록 TPMEM을 수정했다. 하지만 앞서 설명했듯이, 열방향 읽기가 진행되면서 새로운 데이터가 행 방향으로 들어오면 두번째 열 방향 읽기

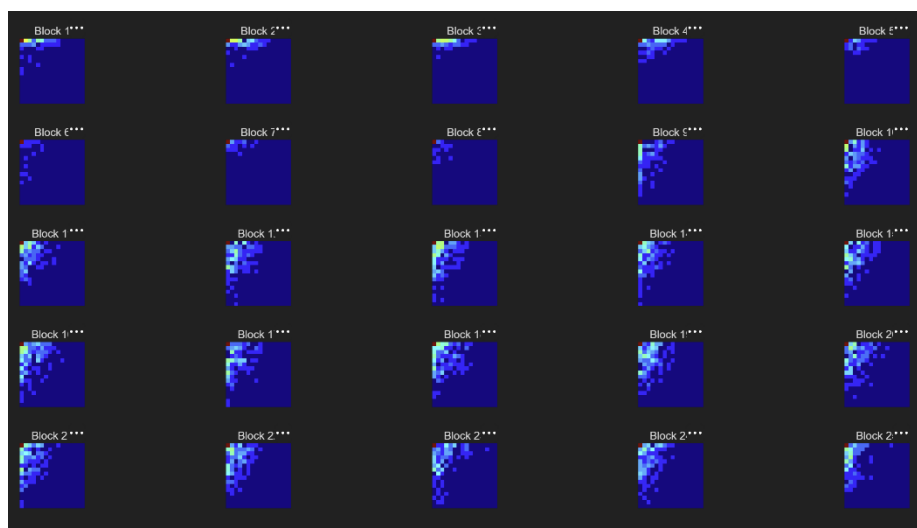
부터는 덮어쓰워진 데이터가 출력될 것이다. 따라서 새로운 스케줄링 알고리즘이 필요하다.



지금까지 설명한 문제는 읽기와 쓰기의 방향이 다르기 때문에 발생한다. 데이터를 전치해서 읽어들이되, 현재 클럭 사이클에서 데이터를 같은 방향으로 읽고 쓰기 위해서는 TPMEM 내부의 카운터를 이용해 phase를 나눠 row-wise 읽기/쓰기와 column-wise 읽기/쓰기를 번갈아가며 진행하면 된다. Phase 0에서는 행 방향으로 데이터를 쓰고, phase 1에서는 열방향으로 데이터를 읽고, 쓰게 된다. 이렇게 하면 이미 읽은 자리에 새로운 데이터가 들어오기 때문에 다음에 읽을 데이터가 손상되지 않는다. Phase 1이 완료되면 다시 행 방향으로 읽기와 쓰기를 반복할 수 있다. 이런 스케줄링을 통해서 데이터를 덮어쓰지도 않고, 매 클럭마다 새로운 데이터를 받을 수 있으며, 이전 phase의 데이터를 전치해서 읽을 수 있게 된다.

새로운 스케줄링 알고리즘을 통해 TPMEM의 개수를 총 4개에서 2개로 줄일 수 있었다.

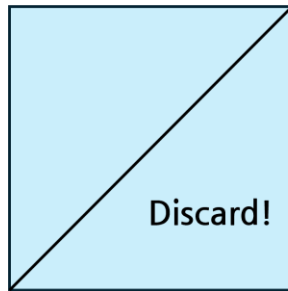
ii. Zero-substitution of High Frequency Signals



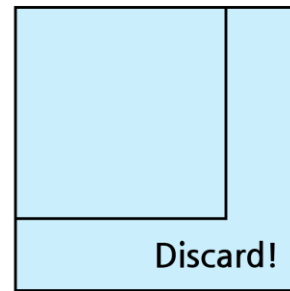
매트랩에서 quantization을 적용한 후 각 블록마다 어떤 값이 분포해 있는지 히트맵으로

나타냈다. 짙은 파란색으로 되어 있는 부분은 0으로 양자화되었다는 것을 의미한다. 어차피 양자화 이후에 값이 0이 될 것이라면, DCT 모듈에서 애초에 계산을 하지 않고 0으로 처리해도 된다는 의미이다. 따라서 0이 될 부분을 DCT 모듈과 TPMEM에서 전부 제외시켜버리고 최종 메모리에 인가해줄 때만 0으로 padding해주면 된다.

Diagonal TPMEM2



Square TPMEM2



고주파수 영역을 0으로 처리하기 위해 TPMEM2를 대각선 방향으로 나누어 아랫부분을 버리는 방식과 정사각형을 유지하되 크기를 줄이는 방식을 비교했다. 아래의 표에서 볼 수 있듯이 대각선 방향으로 절삭하는 것이 면적 감소율은 조금 더 높지만, MATLAB으로 시뮬레이션을 돌려본 결과, PSNR이 비교적 낮게 나왔다. 또한 삼각형 모양의 TPMEM은 제어하거나 RTL로 설계하기 조금 더 복잡할 수 있기 때문에 정사각형 모양을 유지하는 두번째 방식을 채택했다. 이 방법으로 16*16 형태인 TPMEM2를 12*12로 줄였다. DCT2에서 12*1 벡터를 만들게 되면, DCT1에서 만들어지는 16*1 벡터의 하위 4개 원소는 불필요해진다는 것을 알 수 있다. 따라서 DCT1에서도 12*1 벡터를 만들고, 계수행렬과의 모양만 맞추기 위해 0으로 패딩을 하여 12개의 16*1 벡터를 TPMEM1에 저장할 수 있도록 했다. 버려진 TPMEM2의 외곽에 들어갈 값을 생성하는 부분을 DCT 모듈에서도 없앨 수 있었으며, 이로 인해 추가적인 면적 감소를 예상할 수 있다.

	Diagonal	Square
# of Blocks	136 Blocks	144 Blocks
Non-combinational Area Reduction	46.875%	43.75%
Characteristics	• Worse PSNR • Potentially harder to schedule and code	• Better PSNR • Intuitive scheduling

iii. Synthesis-aware RTL Optimization

TPMEM scaling을 통해 TPMEM을 최적화한 후에도 생각보다 면적이 많이 줄지 않았다. 특히 TPMEM2의 면적이 1,059,715 μm^2 로 예상보다 면적 감소율이 적었다. TPMEM2의 면

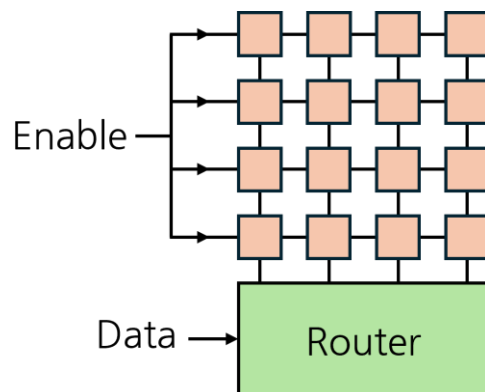
적을 살펴보면 조합논리 회로가 $691,625\mu m^2$, 비조합논리 회로가 $367,670\mu m^2$ 이었다. 레지스터가 차지하는 면적이 주를 이룰 것이라는 생각과는 다른 결과였고, 이 때문에 합성 후 Design Compiler가 만드는 gate-level netlist를 살펴보았다. 이를 통해 array_reg의 Q 포트에 데이터가 입력될 때까지 거쳐가는 게이트가 생각보다 많았고, 입력신호의 fanout이 매우 컸다. 이 사실을 통해서 입력 데이터가 매우 array_reg에 입력되기까지 많은 조합논리 회로를 거쳐간다는 사실을 알 수 있었다.

```

else if (i_enable) begin
    // Write only when index is valid
    if (index < SIZE) begin
        // MSB of counter == 0: Row-wise write
        if (counter[4] == 1'b0) begin
            array[index] <= i_data;
        end
        // MSB of counter == 1: column-wise write
        else begin
            for (row = 0; row < SIZE; row = row + 1) begin
                array[row][(SIZE-index)*BW-1 -: BW] <= i_data[(SIZE-row)*BW-1 -: BW];
            end
        end
    end
end
end

```

이 베릴로그 코드를 보면 i_enable이라는 신호로 모든 array에 동일한 enable 신호를 인가하고 있고, array를 index라는 변수로 인덱싱하고 있는 것을 볼 수 있다. 즉, 아래 그림과 같은 모습으로 합성됐을 것이라고 추정할 수 있다.



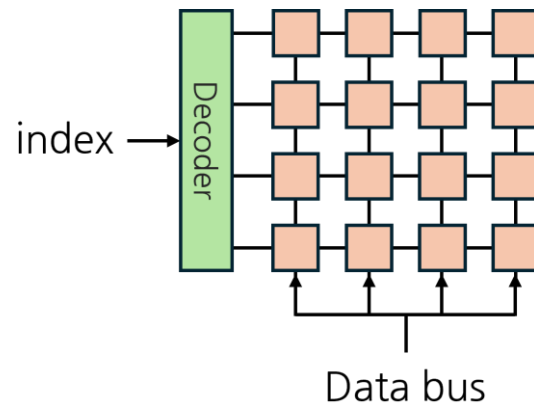
Enable 신호가 모든 FF에 동일하게 입력되므로, 데이터는 자신이 가야할 적절한 FF를 찾아 가야 한다. 이를 위해 거대한 데이터 라우터 회로가 합성됐을 것이다. 데이터는 TPMEM2에서 144비트이기 때문에 라우터의 면적이 커졌을 것이다.

```

else if (i_enable && index < SIZE) begin
    // MSB of counter == 0: Row-wise write
    // Access the array with the variable 'c'
    // instead of the dynamic register value 'index'
    if (counter[4] == 1'b0 && index == r) begin
        array[r][(SIZE-c)*BW-1 -: BW] <= i_data[(SIZE-c)*BW-1 -: BW];
    end
    // MSB of counter == 1: column-wise write
    // Access the array with the variable 'c'
    // instead of the dynamic register value 'index'
    else if (counter[4] == 1'b1 && index == c) begin
        array[r][(SIZE-c)*BW-1 -: BW] <= i_data[(SIZE-r)*BW-1 -: BW];
    end
end
end

```

한편 위의 코드처럼 always @(posedge clk) 내부의 if문에 index 조건을 쓰게 되면, 모든 FF에 동일한 enable 신호가 인가되지 않고 index에 따라 적절한 FF에만 clock enable 혹은 chip select 신호가 인가될 것이라고 예상할 수 있다. 이는 Verilog HDL을 작성하는 정석적인 방법이다.



앞서 설명한 방식으로 코드를 짜게 되면 위의 그림과 같이 일반적으로 자주 다뤄지는 주소 디코더가 합성될 것이라고 기대할 수 있다. 주소 디코더에 들어가는 index는 4비트이기 때문에 144비트 라우터보다 훨씬 면적이 적을 것이라고 예상할 수 있다. Design Compiler가 게이트 단위로 합성을 하기 때문에 실제로 주소 디코더가 합성됐을지는 명확하게 알 수 없지만, 면적을 비교해서 가설을 검증할 수 있다.

```
// Adder shift tree
assign m0 = (s4_0 + s4_1 + s4_2 + s4_3) << 3;
// Adder shift tree
// Explicitly use parentheses to balance the adder tree
// Group the summations by the number of shifts
assign m0 = ((s4_0 + s4_3) + (s4_1 + s4_2)) << 3;
```

또한 위의 오른쪽 그림처럼 DCT 모듈의 코드에서 괄호를 적절하게 배치해 Design Compiler가 균형 잡힌 애더 트리를 형성하도록 유도해주었다. 작은 변화이지만 DCT 모듈 내에 이런 덧셈이 무수히 많이 존재하기 때문에 실제 면적 감소로 이어질 수 있을 것이라고 기대할 수 있다.

	Before		After	
	Combinational	Non-combinational	Combinational	Non-combinational
TPMEM1	443843	505471	379653	505355
TPMEM2	691625	367670	302414	370258
DCT_2D	442516	-	398334	-

위 표는 RTL 수준의 최적화 적용 전과 후 모듈별 면적 변화를 나타낸 것이다. TPMEM에서 non-combinational 면적은 거의 비슷하지만 combinational 면적이 유의미하게 줄어든 것을 볼 수 있다. RTL을 어떻게 작성하는지에 따라서 같은 기능을 수행하더라도 완전히 다르게 합성되는 것을 확인할 수 있다.

iv. Bit Width Reduction

Bit Width	C8 BW 12	C8 BW10	C6 BW12	C6 BW10	C6 BW9
Minimum PSNR	30.11587	29.90643	29.96290	29.77558	29.01205

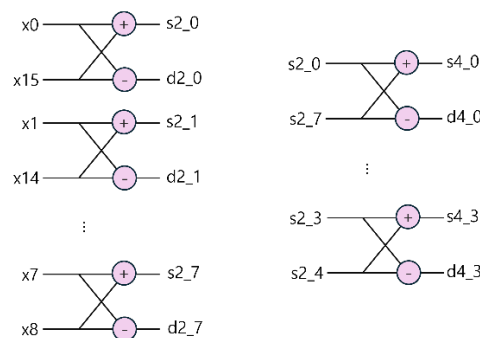
MATLAB으로 계수와 DCT1의 출력 비트너비를 변경해가며 PSNR이 어떻게 변하는지 확인해봤다. 위의 표가 여러 비트너비 조합에 따른 최소 PSNR을 나타낸다. 계수를 6비트로 하고, BW를 10으로 했을 때 PSNR이 적절한 수준으로 유지되며 DCT 모듈과 TPMEM 모두 비트너비 변화의 영향을 받게 되기 때문에 이 비트너비로 설계를 진행했다.

v. Multiplier Removal

$$\begin{array}{lcl}
 c_1 = 0.10\bar{1}0\bar{1} & & X[n] \ll 4 - X[n] \ll 2 - X[n] \\
 c_2 = 0.10\bar{1}0\bar{1} & & X[n] \ll 4 - X[n] \ll 2 - X[n] \\
 c_3 = 0.10\bar{1}0\bar{1} & & X[n] \ll 4 - X[n] \ll 2 - X[n] \\
 c_4 = 0.01010 & & X[n] \ll 3 + X[n] \ll 1 \\
 c_5 = 0.01010 & & X[n] \ll 3 + X[n] \ll 1 \\
 c_6 = 0.01001 & & X[n] \ll 3 + X[n] \\
 c_7 = 0.01001 & & X[n] \ll 3 + X[n] \\
 c_8 = 0.01000 & \times X[n] = & X[n] \ll 3 \\
 c_9 = 0.0100\bar{1} & & X[n] \ll 3 - X[n] \\
 c_{10} = 0.010\bar{1}0 & & X[n] \ll 3 - X[n] \ll 1 \\
 c_{11} = 0.00101 & & X[n] \ll 2 + X[n] \\
 c_{12} = 0.00100 & & X[n] \ll 2 \\
 c_{13} = 0.0010\bar{1} & & X[n] \ll 2 - X[n] \\
 c_{14} = 0.00010 & & X[n] \ll 1 \\
 c_{15} = 0.00001 & & X[n]
 \end{array}$$

처음에는 DCT 모듈을 곱셈기로 설계했지만, 면적을 줄이기 위해 곱셈기를 없애고 Shift adder tree로 합성되도록 RTL을 수정했다. 6비트 계수들을 CSD로 바꾸면 위의 그림과 같이 된다. 계수와 임의의 입력 $X[n]$ 과의 곱셈을 오른쪽처럼 shift와 adder로 변경할 수 있게 된다.

vi. Computation Sharing



DCT의 계수행렬은 좌우가 대칭적이다. 계수 행렬의 짝수 행은 좌우가 완전히 대칭이고,

홀수 행은 좌우가 반대칭(부호가 반대)이다. 따라서 입력 벡터와 계수의 각 행을 곱하는 연산은 입력의 합이나 차를 각각의 계수와 곱하는 연산과 동일하게 처리할 수 있다.

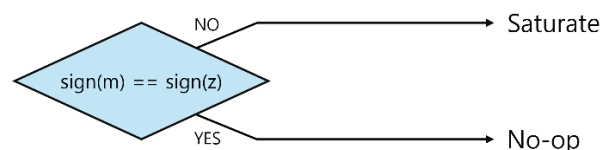
예를 들어 짝수 행에서는 x_0 과 x_{15} 가 항상 같은 계수와 곱해지므로 x_0+x_{15} 를 미리 계산하고 모든 짝수 행에 대해서 재사용할 수 있다. 한편 홀수 행에서는 x_0 과 $-x_{15}$ 가 항상 같은 계수와 곱해진다. 마찬가지로 x_0-x_{15} 를 미리 계산하여 모든 홀수 행에 대해서 재사용할 수 있다. 위의 그림처럼 버터플라이 구조로 입력들의 합과 차를 구한 뒤 이 값을 재사용하게 명시적으로 RTL을 설계함으로써 DCT의 면적을 더 줄일 수 있을 것이다.

$$\begin{aligned}
 c_1 &= 0.10\bar{1}0\bar{1} \\
 c_3 &= 0.10\bar{1}0\bar{1} \\
 c_5 &= 0.010\bar{1}0 \\
 c_{11} &= 0.00\bar{1}0\bar{1}
 \end{aligned}$$

$$\begin{aligned}
 X[n] &\ll 4 - (X[n] \ll 2 + X[n]) \\
 X[n] &\ll 4 - X[n] \ll 2 - X[n] \\
 X[n] &\ll 4 - (X[n] \ll 2 + X[n]) \\
 X[n] &\ll 3 + X[n] \ll 1 \\
 (X[n] \ll 2 + X[n]) &\ll 1 \\
 X[n] &\ll 3 + X[n] \\
 X[n] &\ll 3 + X[n] \\
 X[n] &\ll 3 \\
 X[n] &\ll 3 - X[n] \\
 X[n] &\ll 3 - X[n] \ll 1 \\
 (X[n] \ll 2 + X[n]) & \\
 X[n] &\ll 2 \\
 X[n] &\ll 2 - X[n] \\
 X[n] &\ll 1 \\
 X[n] &
 \end{aligned}$$

더 나아가 홀수 계수들에서 반복되는 비트 패턴 101을 찾을 수 있었다. DCT 연산에서 x_1-x_{14} 과 c_1, c_3, c_5, c_{11} 이 곱해지는 연산이 등장하는데, 공통 비트 패턴과의 곱셈을 미리 계산해주면 그 결과를 4번 재사용할 수 있기 때문에 shift adder의 개수를 더 줄일 수 있을 것이다.

3. Preventing Glitches



글리치가 발생하는 원인은 DCT2 모듈에서 계산 결과를 truncate할 때 MSB를 잃어버려 원래의 결과와 부호가 달라질 수 있기 때문이다. 음수였던 값이 양수로 바뀌거나 양수였던 값이 음수로 바뀌면 어두운 부분이 밝게 변하거나, 밝은 부분이 어둡게 변하는 등의 현상이 일어날 수 있다. 이는 블록의 좌측 최상단의 DC 신호에서 발생할 경우 더욱 두드러질 수 있다. 이를 해결하기 위해서 DCT2의 연산 결과인 m 과 truncation 결과인 z 의 부호를 비교하여, 부호가 다르면 z 가 표현할 수 있는 가장 크거나 작은 수로 포화시키는 방법을 적용하여 해결했다. 위의 control flow가 이 방법을 잘 보여준다. 만약 부호가 같다면 아무것도 적용하지 않고 truncate을 하면 된다.

```
// Overflow compensation
assign z0c = ({m0[IN_BW+8], z0[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m0[IN_BW+8], z0[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z0[OUT_BW-1:0];
assign z1c = ({m1[IN_BW+8], z1[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m1[IN_BW+8], z1[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z1[OUT_BW-1:0];
assign z2c = ({m2[IN_BW+8], z2[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m2[IN_BW+8], z2[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z2[OUT_BW-1:0];
assign z3c = ({m3[IN_BW+8], z3[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m3[IN_BW+8], z3[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z3[OUT_BW-1:0];
assign z4c = ({m4[IN_BW+8], z4[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m4[IN_BW+8], z4[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z4[OUT_BW-1:0];
assign z5c = ({m5[IN_BW+8], z5[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m5[IN_BW+8], z5[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z5[OUT_BW-1:0];
assign z6c = ({m6[IN_BW+8], z6[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m6[IN_BW+8], z6[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z6[OUT_BW-1:0];
assign z7c = ({m7[IN_BW+8], z7[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m7[IN_BW+8], z7[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z7[OUT_BW-1:0];
assign z8c = ({m8[IN_BW+8], z8[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m8[IN_BW+8], z8[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z8[OUT_BW-1:0];
assign z9c = ({m9[IN_BW+8], z9[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m9[IN_BW+8], z9[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z9[OUT_BW-1:0];
assign z10c = ({m10[IN_BW+8], z10[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m10[IN_BW+8], z10[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z10[OUT_BW-1:0];
assign z11c = ({m11[IN_BW+8], z11[OUT_BW-1]} == 2'b01 ? 12'b0111_1111_1111 : ({m11[IN_BW+8], z11[OUT_BW-1]} == 2'b10 ? 12'b1000_0000_0000 : z11[OUT_BW-1:0];
```

위의 코드처럼 베릴로그를 작성해주어 글리치를 통제할 수 있었다.

4. Control Flow and Architecture

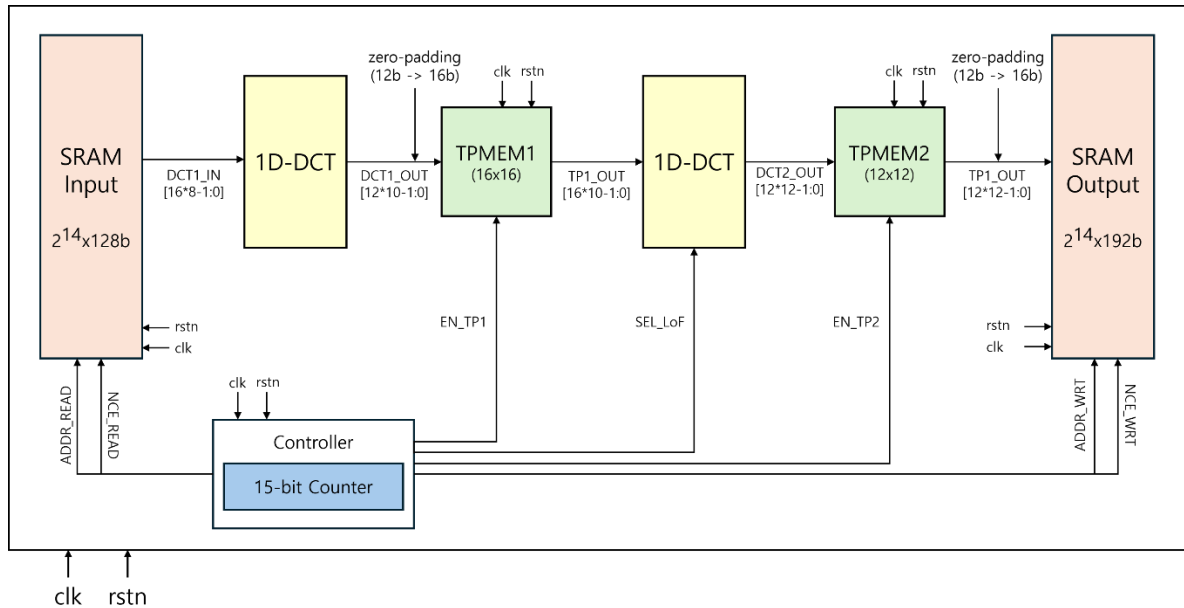
i. Control Flow

cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
RUN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
COUNT	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
NCE_READ	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
ADDR_READ	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
MEM1_OUT	X0	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	
DCT1_IN	X0	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	
DCT1_OUT	Z0	Z1	Z2	Z3	Z4	Z5	Z6	Z7	Z8	Z9	Z10	Z11	Z12	Z13	Z14	Z15	Z16	
EN_TP1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
TPMEM1_Array		Z0	Z1	Z2	Z3	Z4	Z5	Z6	Z7	Z8	Z9	Z10	Z11	Z12	Z13	Z14	Z15	
TPMEM1_OUT																		
DCT2_IN																		
EN_TP2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DCT2_OUT																		
TPMEM2_Array																		
TPMEM2_OUT																		
NCE_WRT	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
ADDR_WRT	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
sel	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
MEM2_Data																		

cycle	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
RUN	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
COUNT	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
NCE_READ	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
ADDR_READ	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
MEM1_OUT	X17	X18	X19	X20	X21	X22	X23	X24	X25	X26	X27	X28	X29	X30	X31	X32	X33	X34	X35	X36	X37	X38	X39
DCT1_IN	X17	X18	X19	X20	X21	X22	X23	X24	X25	X26	X27	X28	X29	X30	X31	X32	X33	X34	X35	X36	X37	X38	X39
DCT1_OUT	Z17	Z18	Z19	Z20	Z21	Z22	Z23	Z24	Z25	Z26	Z27	Z28	Z29	Z30	Z31	Z32	Z33	Z34	Z35	Z36	Z37	Z38	Z39
EN_TP1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
TPMEM1_Array	Z17	Z18	Z19	Z20	Z21	Z22	Z23	Z24	Z25	Z26	Z27	Z28	Z29	Z30	Z31	Z32	Z33	Z34	Z35	Z36	Z37	Z38	Z39
TPMEM1_OUT	Z0_T	Z1_T	Z2_T	Z3_T	Z4_T	Z5_T	Z6_T	Z7_T	Z8_T	Z9_T	Z10_T	Z11_T	Z12_T	Z13_T	Z14_T	Z15_T	Z16_T	Z17_T	Z18_T	Z19_T	Z20_T	Z21_T	Z22_T
DCT2_IN	Z0_T	Z1_T	Z2_T	Z3_T	Z4_T	Z5_T	Z6_T	Z7_T	Z8_T	Z9_T	Z10_T	Z11_T	Z12_T	Z13_T	Z14_T	Z15_T	Z16_T	Z17_T	Z18_T	Z19_T	Z20_T	Z21_T	Z22_T
EN_TP2	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
DCT2_OUT	Z0'	Z1'	Z2'	Z3'	Z4'	Z5'	Z6'	Z7'	Z8'	Z9'	Z10'	Z11'	Z12'	Z13'	Z14'	Z15'	Z16'	Z17'	Z18'	Z19'	Z20'	Z21'	Z22'
TPMEM2_Array	Z0'	Z1'	Z2'	Z3'	Z4'	Z5'	Z6'	Z7'	Z8'	Z9'	Z10'	Z11'	Z12'	Z13'	Z14'	Z15'	Z16'	Z17'	Z18'	Z19'	Z20'	Z21'	Z22'
TPMEM2_OUT																							
NCE_WRT	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
ADDR_WRT	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
sel	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
MEM2_Data																							

컨트롤 모듈을 설계하기 위해 위와 같은 datapath 및 제어 신호 흐름을 완성했다. DCT 모듈들은 순수 조합회로만 있기 때문에 한 클럭 사이클 내에 연산이 완료되고, TPMEM 최적화를 통해 모든 사이클에서 TPMEM에 쓰기와 읽기가 가능하다. 초록색으로 칠해진 신호는 컨트롤 모듈에서 인가하는 제어신호이고, 보라색으로 칠해진 칸은 데이터의 흐름이다.

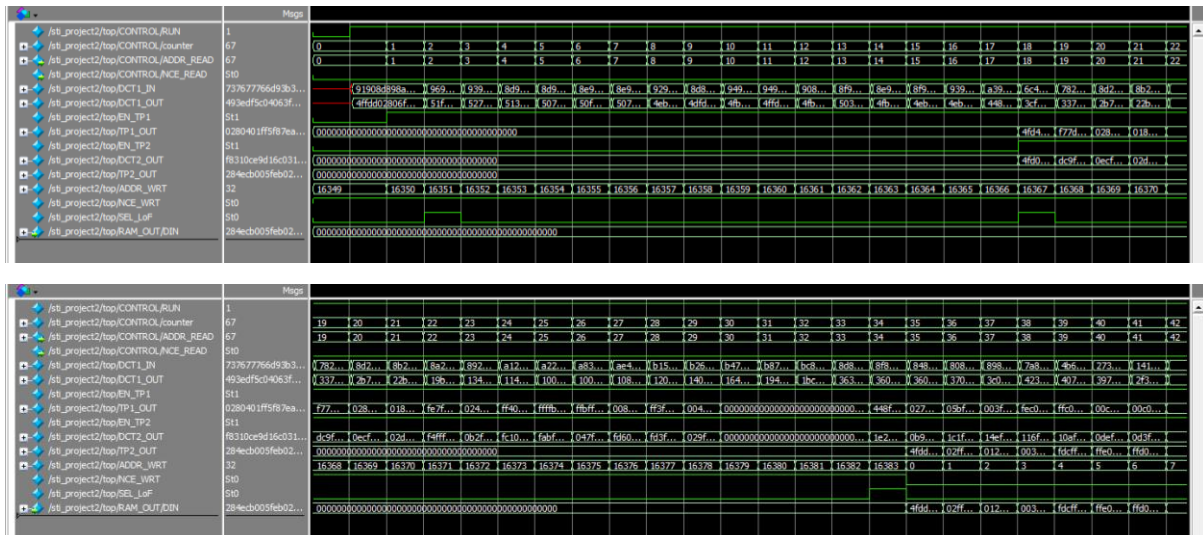
ii. Architecture



지금까지 논의한 설계 방법을 적용한 최종적인 아키텍처이다.

5. Results

i. Waveform



위의 그림은 시뮬레이션 결과이고, 각 제어신호와 데이터의 흐름이 의도한 대로 출력되는 것을 확인할 수 있다. 파이프라인이 포화되어 TPMEM2에서 데이터를 뱃을 때까지 36 사이클이 걸리고(36번째 사이클에서 데이터가 메모리에 인가되고 37번째에서 메모리에 저장된다), 그 이후에는 1사이클마다 16*1 벡터가 출력되고, 16사이클마다 하나의 16*16 블록이 출력된다.

ii. PSNR



모든 사진의 PSNR이 29를 초과한다. 글리치가 발생하는 사진도 없다.

iii. Synthesis

Startpoint: TPMEM1/o_data_reg[0]			DCT_2D/r870/U1_5/OUTC (fadd1s3)	0.37	4.73 f
(rising edge-triggered flip-flop clocked by clk)			DCT_2D/r870/U1_6/OUTC (fadd1s2)	0.46	5.19 f
Endpoint: TPMEM2/array_reg[0][130]			DCT_2D/r870/U1_7/OUTC (fadd1s2)	0.47	5.67 f
(rising edge-triggered flip-flop clocked by clk)			DCT_2D/r870/U1_8/OUTC (fadd1s2)	0.47	6.14 f
Path Group: clk			DCT_2D/r870/U1_9/OUTC (fadd1s2)	0.45	6.59 f
Path Type: max			DCT_2D/r870/U1_10/OUTC (fadd1s1)	0.50	7.09 f
			DCT_2D/r870/U1_11/OUTC (fadd1s2)	0.83	7.92 r
			DCT_2D/r870/SUM[11] (DCT2_DW01_add_20)	0.00	7.92 r
Point	Incr	Path	DCT_2D/U194/Q (nb1s4)	0.23	8.15 r
-----			DCT_2D/add_106_3/A[11] (DCT2_DW01_add_2)	0.00	8.15 r
clock clk (rise edge)	0.00	0.00	DCT_2D/add_106_3/U1_11/OUTC (fadd1s3)	0.33	8.48 r
clock network delay (ideal)	0.00	0.00	DCT_2D/add_106_3/U1_12/OUTC (fadd1s3)	0.34	8.82 r
TPMEM1/o_data_reg[0]/CLK (dffs2)	0.00 #	0.00 r	DCT_2D/add_106_3/U1_13/Q (xor3s1)	0.60	9.42 r
TPMEM1/o_data_reg[0]/Q (dffs2)	0.32	0.32 f	DCT_2D/add_106_3/SUM[13] (DCT2_DW01_add_2)	0.00	9.42 r
TPMEM1/o_data[0] (TPMEM1)	0.00	0.32 f	DCT_2D/U326/Q (nnd2s2)	0.10	9.52 f
DCT_2D/X[0] (DCT2)	0.00	0.32 f	DCT_2D/U202/Q (oi21s3)	0.23	9.75 r
DCT_2D/add_66/B[0] (DCT2_DW01_add_26)	0.00	0.32 f	DCT_2D/U268/Q (tbls1)	0.16	9.91 f
DCT_2D/add_66/U7/Q (and2s1)	0.43	0.74 f	DCT_2D/U321/Q (nnd3s2)	0.19	10.10 r
DCT_2D/add_66/U1_1/OUTS (fadd1s3)	0.93	1.67 r	DCT_2D/U320/Q (nnd2s2)	0.15	10.24 f
DCT_2D/add_66/SUM[1] (DCT2_DW01_add_26)	0.00	1.67 r	DCT_2D/U200/Q (nor2s2)	0.24	10.48 r
DCT_2D/add_84/A[1] (DCT2_DW01_add_24)	0.00	1.67 r	DCT_2D/U167/Q (i1s5)	0.14	10.53 f
DCT_2D/add_84/U1_1/OUTS (fadd1s3)	0.73	2.40 f	DCT_2D/U295/Q (oi221s2)	0.41	11.03 r
DCT_2D/add_84/SUM[1] (DCT2_DW01_add_24)	0.00	2.40 f	DCT_2D/Z[142] (DCT2)	0.00	11.03 r
DCT_2D/r870/A[1] (DCT2_DW01_add_20)	0.00	2.40 f	TPMEM2/i_data[142] (TPMEM2)	0.00	11.03 r
DCT_2D/r870/U1_1/OUTC (fadd1s1)	0.54	2.94 f	TPMEM2/U52/Q (nnd2s2)	0.44	11.47 f
DCT_2D/r870/U1_2/OUTC (fadd1s1)	0.47	3.41 f	TPMEM2/U150/Q (oi22s2)	0.50	11.98 r
DCT_2D/r870/U1_3/OUTC (fadd1s1)	0.54	3.95 f	TPMEM2/array_reg[0][130]/DIN (dffles1)	0.00	11.98 r
DCT_2D/r870/U1_4/OUTC (fadd1s3)	0.41	4.36 f	data arrival time		11.98
DCT_2D/r870/U1_5/OUTC (fadd1s3)	0.37	4.73 f	clock clk (rise edge)	12.50	12.50
DCT_2D/r870/U1_6/OUTC (fadd1s2)	0.46	5.19 f	clock network delay (ideal)	0.00	12.50
DCT_2D/r870/U1_7/OUTC (fadd1s2)	0.47	5.67 f	TPMEM2/array_reg[0][130]/CLK (dffles1)	0.00	12.50 r
DCT_2D/r870/U1_8/OUTC (fadd1s2)	0.47	6.14 f	library setup time	-0.52	11.98
DCT_2D/r870/U1_9/OUTC (fadd1s2)	0.45	6.59 f	data required time		11.98
DCT_2D/r870/U1_10/OUTC (fadd1s1)	0.50	7.09 f	data required time		11.98
DCT_2D/r870/U1_11/OUTC (fadd1s2)	0.83	7.92 r	data arrival time		11.98
DCT_2D/r870/SUM[11] (DCT2_DW01_add_20)	0.00	7.92 r	-----		
DCT_2D/U194/Q (nb1s4)	0.23	8.15 r	slack (MET)		0.00

12.5ns 클럭으로 합성했을 때의 타이밍 리포트이다. 타이밍 조건을 만족한 상황이며, 크리티컬 패스는 TPMEM1에서 출발해 두번째 DCT 모듈을 거쳐 TPMEM2로 도착하는 경로이다. 첫번째 DCT 모듈보다 입출력 비트너비가 넓기 때문에 두번째 DCT 모듈이 지연이 더 클 것이고 타이밍 리포트를 통해서 확인할 수 있다. 만약 더 빠른 클럭에서 동작해야

한다면 DCT 모듈 중간에 파이프라인 레지스터를 추가하는 것을 고려해볼 수 있다.

Hierarchical area distribution					DCT_2D	398334.0625	16.9	43562.3281	0.0000
					DCT_2D/add_0_root_add_112_4	2844.9812	0.1	2844.9812	0.0000
					DCT_2D/add_0_root_add_113_2	2728.8594	0.1	2728.8594	0.0000
					DCT_2D/add_0_root_add_123_2	2670.7971	0.1	2670.7971	0.0000
					DCT_2D/add_106_3	2538.0872	0.1	2538.0872	0.0000
					DCT_2D/add_109_2	2471.7312	0.1	2471.7312	0.0000
					DCT_2D/add_112_2	2139.9551	0.1	2139.9551	0.0000
					DCT_2D/add_112_6	3126.9912	0.1	3126.9912	0.0000
					DCT_2D/add_112_7	2820.0981	0.1	2820.0981	0.0000
					DCT_2D/add_114_4	2123.3660	0.1	2123.3660	0.0000
					DCT_2D/add_114_3	3226.5237	0.1	3226.5237	0.0000
					DCT_2D/add_114_4	3052.3406	0.1	3052.3406	0.0000
					DCT_2D/add_117_1	1974.0671	0.1	1974.0671	0.0000
					DCT_2D/add_117_3	1957.4780	0.1	1957.4780	0.0000
					DCT_2D/add_117_4	2139.9551	0.1	2139.9551	0.0000
					DCT_2D/add_117_5	2770.3313	0.1	2770.3313	0.0000
					DCT_2D/add_117_6	2455.1418	0.1	2455.1418	0.0000
					DCT_2D/add_117_7	2206.3113	0.1	2206.3113	0.0000
					DCT_2D/add_117_8	3317.7615	0.1	3317.7615	0.0000
					DCT_2D/add_120_1	1974.0671	0.1	1974.0671	0.0000
					DCT_2D/add_120_2	3342.6450	0.1	3342.6450	0.0000
					DCT_2D/add_120_3	2048.7170	0.1	2048.7170	0.0000
					DCT_2D/add_120_4	2952.8074	0.1	2952.8074	0.0000
					DCT_2D/add_120_5	3326.0566	0.1	3326.0566	0.0000
					DCT_2D/add_123_1	2139.9551	0.1	2139.9551	0.0000
					DCT_2D/add_123_2	2015.5391	0.1	2015.5391	0.0000
					DCT_2D/add_123_4	2737.1526	0.1	2737.1526	0.0000
					DCT_2D/add_123_5	3292.8792	0.1	3292.8792	0.0000
					DCT_2D/add_123_6	3608.0662	0.2	3608.0662	0.0000
					DCT_2D/add_126_1	2131.6611	0.1	2131.6611	0.0000
					DCT_2D/add_126_2	2372.1990	0.1	2372.1990	0.0000
					DCT_2D/add_126_3	4379.4463	0.2	4379.4463	0.0000
					DCT_2D/add_126_4	2961.1018	0.1	2961.1018	0.0000
					DCT_2D/add_126_5	3077.2244	0.1	3077.2244	0.0000
					DCT_2D/add_129_1	2123.3660	0.1	2123.3660	0.0000
					DCT_2D/add_129_2	2480.0259	0.1	2480.0259	0.0000
					DCT_2D/add_129_3	1957.4780	0.1	1957.4780	0.0000
					DCT_2D/add_129_4	3724.1880	0.2	3724.1880	0.0000
					DCT_2D/add_129_5	4454.0918	0.2	4454.0918	0.0000
					TPMEM1	885424.9375	37.6	379653.4375	505355.4375
					TPMEM1/add_32	414.7200	0.0	414.7200	0.0000
					TPMEM2	673081.5625	28.6	302414.1562	370258.7188
					TPMEM2/add_27	414.7200	0.0	414.7200	0.0000
					Total	1434460.7500	922842.6250		

Module	Area
CONTROL	6,569.17 μm^2
DCT_1D	336,302.41 μm^2
DCT_2D	398,334.06 μm^2
TPMEM1	885,424.94 μm^2
TPMEM2	673,081.56 μm^2
Total	2,357,478.5 μm^2

report_area -hierarchy를 통해 모든 모듈의 면적을 확인할 수 있었다. 위 표는 각 모듈의 면적을 소수점 셋째자리에서 반올림한 결과를 나타낸 것이다. 최종적으로 탑모듈이 2,357,478.5 μm^2 으로 합성됐다. 요구 조건인 3,000,000 μm^2 보다 훨씬 낮은 면적이다.

```

*****
Report : power
        -analysis_effort low
Design : DCT_TOP
Version: Z-2007.03-SP4
Date   : Mon Jun  8 06:53:24 2026
*****

Library(s) Used:

        lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)

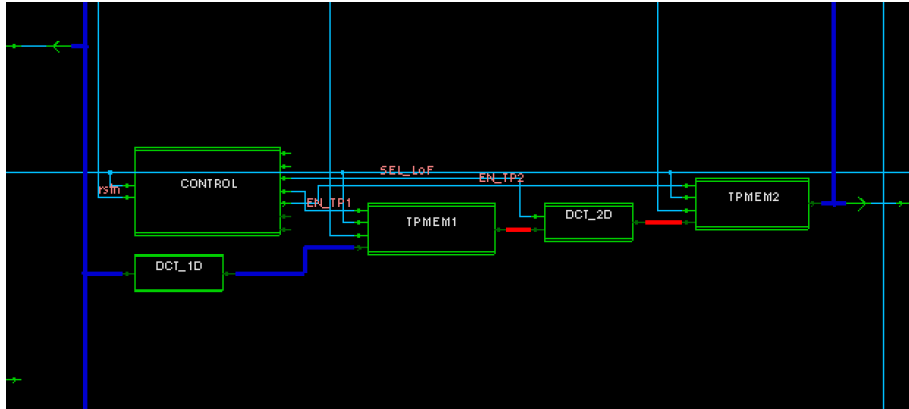
Operating Conditions: nom_pvt   Library: lec25dsc25_SS
Wire Load Model Mode: top

Global Operating Voltage = 2.25
Power-specific unit information :
    Voltage Units = 1V
    Capacitance Units = 1.000000pf
    Time Units = 1ns
    Dynamic Power Units = 1mW (derived from V,C,T units)
    Leakage Power Units = 1pW

Cell Internal Power = 61.1439 mW (79%)
Net Switching Power = 16.0408 mW (21%)
-----
Total Dynamic Power = 77.1847 mW (100%)
Cell Leakage Power  = 3.3953 mW

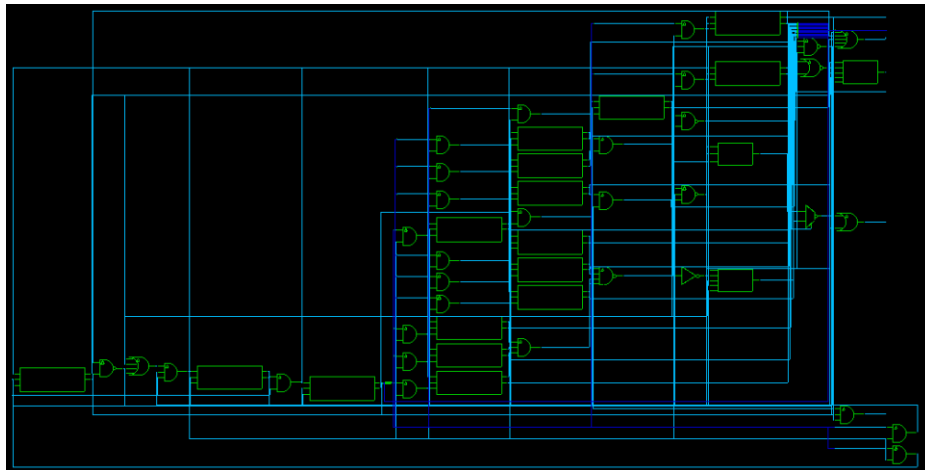
```

파워는 77.1847mW였다. 최적화를 진행하면서 소모되는 파워도 함께 감소했다.



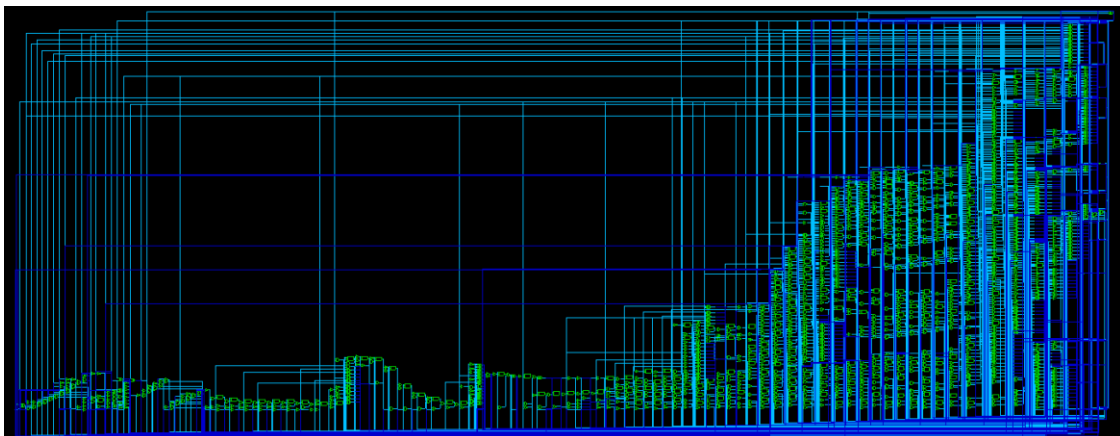
[Top Module]

합성된 탑 모듈을 확대한 모습이고, 모든 모듈이 제대로 연결된 것을 볼 수 있다.
DCT_2D를 통과하는 빨간 경로가 크리티컬 패스이다.

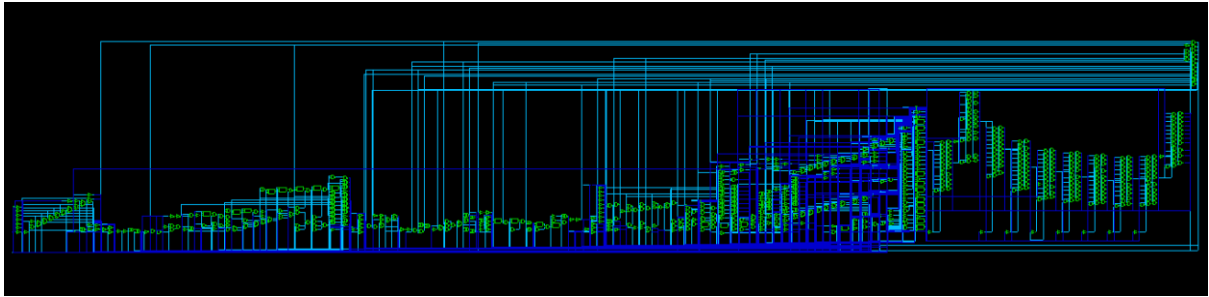


[Control Module]

컨트롤 모듈은 위와 같이 합성됐다. 카운터를 통해 간단하게 제어 신호들을 인가해줬기 때문에 카운터에 쓰일 레지스터들과 간단한 조합회로가 합성된 것을 볼 수 있다.



[DCT_1D]



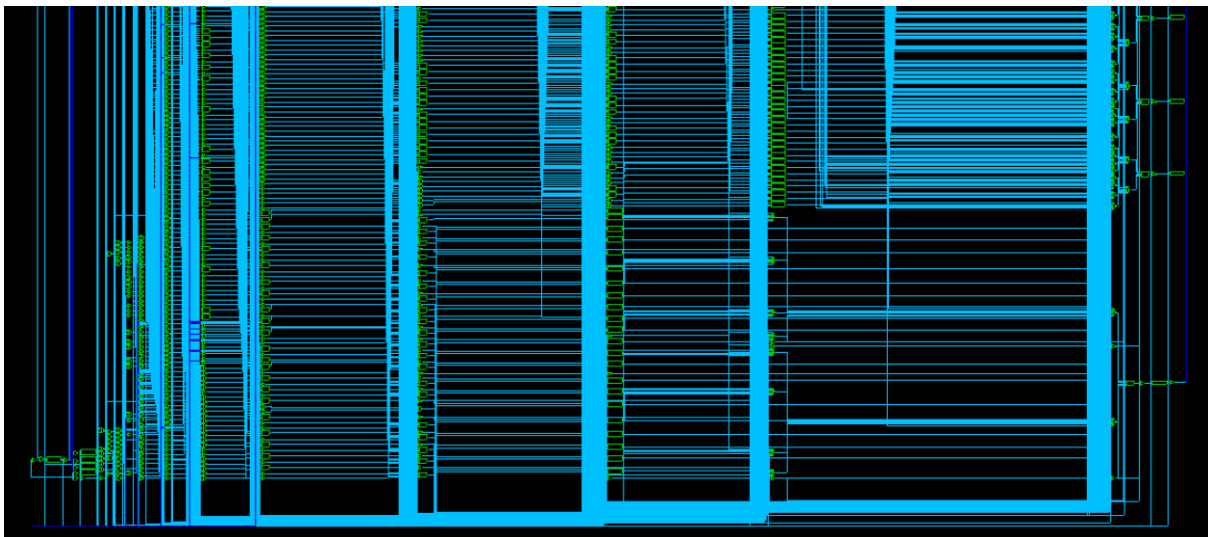
[DCT_2D]

DCT 모듈들은 위와 같이 합성됐다.



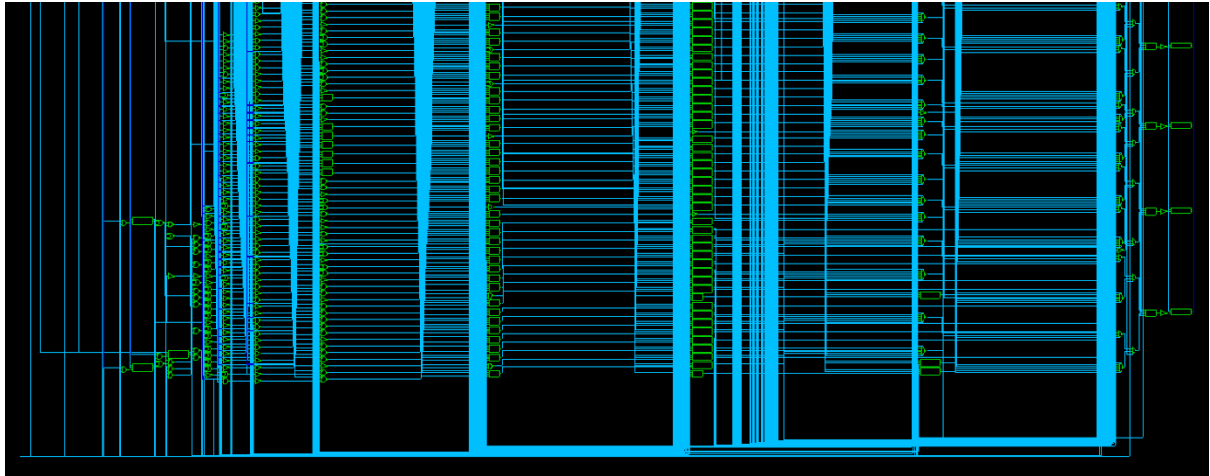
[TPMEM1 and TPMEM2, rotated by 90 degrees]

TPMEM1과 TPMEM2는 위와 같이 수많은 array_reg와 이를 연결해주는 와이어들이 스키매틱을 점유하고 있다.



[TPMEM1]

TPMEM1을 확대하면 index 디코더를 위한 조합회로가 왼쪽에 합성됐고, 나머지는 array_reg와 data bus인 것으로 보인다.



[TPMEM2]

TPMEM2도 TPMEM1과 구조가 거의 동일하다.

6. Discussion

$[\mu m^2]$	Baseline	A	B	C	D
CONTROL	6,635	6,668	6,668	6,602	6,569
TPMEM1	1,217,083	1,090,253	950,275	949,730	885,424
TPMEM2	1,229,399	960,660	964,246	1,059,715	673,081
DCT1	774,404	641,581	415,605	395,841	336,302
DCT2	1,317,255	1,198,368	519,589	442,516	398,334
Total	4,613,235	3,958,130	2,917,245	2,911,900	2,357,478

A: Zero-substitution of High Frequency Signals (TPMEM Scaling)

B: Bit-width Reduction

C: Multiplier Removal and computation sharing

D: Synthesis-aware RTL Optimization (Decoder based TPMEM structure)

위 표는 최초 설계 (TPMEM 2개, C8, BW12, DCT 곱셈기로 구현)를 baseline으로 하고, 각각의 최적화 기법 A, B, C, D를 적용했을 때 면적이 어떻게 변했는지를 추적한 표이다. 최종적으로 컨트롤 모듈을 제외한 모든 모듈에서 상당한 면적 감소를 이뤄냈으며 최종 면적은 $2,357,478 \mu m^2$ (소수점 버림)인 것을 확인할 수 있다. 특히 TPMEM에서 조합회로가 과도하게 사용되는 것을 파악하고 RTL 수준에서 최적화를 적용한 것이 유의미한 면적 감소로 이어진 것으로 보아 TPMEM의 합성 결과에 대한 추정이 맞았다는 것을 확인할 수 있었다.

$[\mu m^2]$	Baseline	A	B	C	D
CONTROL	1	1.005	1.005	0.995	0.990
TPMEM1	1	0.896	0.781	0.780	0.727
TPMEM2	1	0.781	0.784	0.862	0.547
DCT1	1	0.828	0.537	0.511	0.434
DCT2	1	0.910	0.394	0.336	0.302
Total	1	0.858	0.632	0.631	0.511

위 표는 최적화를 적용하지 않은 모듈의 면적을 1로 나타냈을 때, 각 최적화 기법을 적용했을 때 모듈의 면적이 baseline의 몇 배인지를 나타낸 것이다. 각 단계별로 모듈들의 면적이 얼마나 줄었는지 더 직관적으로 확인할 수 있다. DCT 모듈에서는 비트너비 축소가 가장 큰 면적 감소를 이끌어낸 것을 볼 수 있다. 모든 최적화 기법을 적용한 후 최종 면적은 baseline의 51.1%인 것을 볼 수 있다.

이번 프로젝트를 진행하며 회로 설계 이전에 그 회로가 수행할 알고리즘에 대해 정확하게 파악하는 것이 중요하다는 것을 알게 됐다. 알고리즘의 특성을 정확히 알고 이용하는 것이 하드웨어 최적화의 필수 조건인 것을 알게 됐다. 또한 실제 RTL을 작성하는 것보다 회로의 순차적인 동작과 데이터 흐름을 정확히 설계하는 데 훨씬 많은 시간을 쏟아야 했다. TPMEM 스케줄링을 개선한 것처럼 같은 기능을 수행하더라도 데이터 흐름이나 제어를 달리 함으로써 면적을 훨씬 줄일 수 있었다.

또한 HDL을 작성할 때 코딩 스타일에 따라 같은 기능을 수행하더라도 하드웨어 합성 결과가 천지차이라는 것을 실감했다. 합성툴이 코드를 어떻게 다루는지를 알고, 정석적인 HDL 작성법을 따르는 것도 하드웨어 설계를 할 때 중요하다는 것을 알게 됐다.